DEMOKRITOS

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

UNIVERSITY OF PIRAEUS

Deep Learning Image Classification in CIFAR-100 Dataset

Faidon Grammatikopoulos - mtn2505

Konstantinos Dimopoulos - mtn2507

Theodoros Kokkas - mtn2511

Deep Learning Project Report

Course: Deep Learning
MSc in Artificial Intelligence

University of Piraeus & NCSR "Demokritos", Greece

June 2026

Contents

1 Introduction **1**

2 Image Classification Problem and Dataset **3**

2.1 Image Classification 3

2.2 The CIFAR-100 Dataset 3

2.3 Task Types 3

2.4 Class Imbalance and Metric Choice 4

3 Architectures **5**

3.1 Overview 5

3.2 Sequence Models 5

3.2.1 Vanilla RNN 6

3.2.2 LSTM 6

3.2.3 Bidirectional LSTM 6

3.3 From-Scratch CNNs 6

3.3.1 Compact Baseline CNN 6

3.3.2 Stronger Regularised CNN 7

3.4 Transfer-Learning CNNs 8

3.4.1 Fine-Tuning Regimes 8

3.4.2 ResNet50V2 8

3.4.3 DenseNet121 8

3.4.4 EfficientNet 9

3.4.5 MobileNetV3Small 9

3.5 Vision Transformer 9

3.5.1 Hugging Face ViT Baseline 9

4 Experimental Setup **11**

4.1 Dataset Loading and Splits 11

4.2 Local Source-Code Workflow 11

4.3 Colab Notebook Workflow 12

4.4 Metrics 13

4.4.1 Multiclass Metrics 13

4.4.2 Binary Metrics 14
4.5 Reproducibility and Artifacts 14

5 Results and Discussion 16

5.1 Baseline CNN Binary Results 17
5.1.1 Coarse Superclass Binary Tasks 17
5.1.2 Fine Class Binary Tasks 17
5.2 Baseline CNN Multiclass 18
5.3 From-Scratch Controls 19
5.4 Transfer-Learning Multiclass: Coarse Superclasses 20
5.5 Transfer-Learning Multiclass: Fine Classes 21
5.5.1 Architecture Comparison: Macro F1 23
5.6 ResNet and DenseNet Binary Highlights 24
5.7 Vision Transformer 24
5.8 Discussion 25

6 Future Work 27

7 Conclusions 29

Chapter 1

Introduction

Image classification is one of the foundational problems in computer vision and a natural proving ground for deep learning architectures. Rather than optimising a single model on a single task, this project takes a comparative approach: it places several architecture families on common ground and asks how each one performs when the training data, the task definition, and the evaluation protocol are held constant. The shared benchmark is the CIFAR-100 dataset, and the shared theme is binary and multiclass classification of its small, information-dense 32 by 32 RGB images.

CIFAR-100 consists of 60 000 images distributed across 100 fine-grained object categories, which are further grouped into 20 coarse superclasses. This dual label structure supports three complementary task styles. Binary classification casts a single fine class or a single superclass against all remaining images, producing a highly imbalanced but practically relevant scenario. Coarse multiclass classification treats the 20 superclasses as the target space, offering a moderately difficult 20-way problem. Fine multiclass classification uses all 100 categories and presents the hardest setting. Together, the three styles let the comparison span a wide range of difficulty without changing the underlying images.

The architectures studied in this report span four broad families. Sequence models treat each image as a time series of pixel rows and process it with a recurrent network; the variants examined are a vanilla RNN, an LSTM, and a Bi-LSTM, each reading the 32 rows of an image as a sequence of 96-dimensional vectors. From-scratch convolutional networks serve as controls: a compact baseline CNN and a stronger, more regularised CNN establish what a purpose-built image model can achieve without pre-training. Transfer-learning CNNs then ask how much further performance can be pushed by importing a backbone pre-trained on ImageNet; the backbones compared are ResNet, DenseNet, EfficientNet, and MobileNetV3, evaluated both with a frozen feature extractor and with partial fine-tuning. Finally, a Vision Transformer experiment adapted from the Hugging Face model hub tests whether an attention-based architecture competitive on large-scale benchmarks translates to 32 by 32 images with limited compute.

The project is delivered along two parallel tracks. The first is a structured local source-code implementation organised into the `data/`, `models/`, `training/`, `evaluation/`, and `experiments/` packages, driven by YAML configuration files. This track is designed to be modular, testable, and straightforward to extend. The second track consists of standalone Colab notebooks located under `notebooks/`; each notebook reproduces the same workflows self-containedly, without importing the local package, so that the full pipeline

from data loading to evaluation plots can be run on a free Colab T4 GPU. The two tracks are kept conceptually aligned but deliberately decoupled.

The remainder of this report follows the order of the experimental program. Chapter 2 introduces the dataset and the binary and multiclass task construction in more detail. Chapter 3 describes each architecture family and the modelling decisions specific to it. Chapter 4 covers the experimental setup, including preprocessing, augmentation, training protocols, and evaluation metrics. Chapter 5 presents and discusses results. Chapter 6 considers directions for future work, and Chapter 7 concludes.

The implementation can be found under the open repository :
https://github.com/Fgram-devAI/deepl-cifar100-image-analysis

Chapter 2

Image Classification Problem and Dataset

2.1 Image Classification

Image classification is the task of assigning a discrete label, or a set of labels, to a raster image. A model receives a fixed-size pixel array and returns a probability distribution over a predefined label vocabulary. The difficulty of the problem scales with the number of classes, the visual similarity between them, the image resolution, and the size of the labelled training set. Binary classification is the simplest instantiation: the label vocabulary contains exactly two outcomes, and the model learns to separate a target concept from everything else.

2.2 The CIFAR-100 Dataset

CIFAR-100 [1] contains 60 000 colour images at a resolution of 32 by 32 pixels, partitioned into 50 000 training images and 10 000 test images. Each image carries two labels simultaneously. The *fine label* identifies one of 100 object categories such as `cow`, `mushroom`, `snake`, `orange`, or `skyscraper`. The *coarse label* identifies which of 20 superclasses the fine category belongs to; for example, the superclass `aquatic_mammals` groups together beaver, dolphin, otter, seal, and whale under a single semantic umbrella.

The 100 fine classes are distributed uniformly: each class contributes 500 training images and 100 test images. At the superclass level, each of the 20 groups aggregates five fine classes, giving 2 500 training images and 500 test images per superclass. The dual label hierarchy makes CIFAR-100 well suited for studying how task granularity and class imbalance interact with model architecture.

2.3 Task Types

This project defines three complementary task styles over CIFAR-100.

Target-vs-rest binary classification at the fine level. A single fine class is designated the positive class and all remaining images form the negative class. Concretely, a task such as `cow` vs. rest trains a model to detect images of cows among the full dataset. Other

fine-level binary tasks used in this project include `mushroom`, `orange`, `skyscraper`, and `snake` against all other classes.

Target-vs-rest binary classification at the coarse level. A single superclass is designated the positive class and all remaining images form the negative class. For example, the task `aquatic_mammals` vs. rest asks the model to recognise any of the five aquatic mammal categories as a positive instance. Other coarse-level binary tasks include `fish`, `flowers`, `food_containers`, and `people` against all other superclasses.

Multiclass classification. The 20-way coarse problem treats the superclass label as the target, while the 100-way fine problem treats the fine label as the target. These settings are the most natural framing for standard accuracy benchmarking and allow direct comparison with published CIFAR-100 results.

2.4 Class Imbalance and Metric Choice

Binary tasks derived from a single fine class are highly imbalanced. With 500 positive training images out of 50 000 total, the positive class represents approximately 1 % of the training set. A classifier that always predicts the negative class would therefore achieve around 99 % accuracy while providing no useful information about the target concept. Accuracy is consequently a misleading metric for fine-level binary tasks, and results must be interpreted through precision, recall, F1 score, and the area under the precision–recall curve (PR AUC).

Coarse-level binary tasks are less extreme: a single superclass accounts for roughly 5 % of the data (2 500 of 50 000 training images), which still warrants careful metric choice but allows accuracy to carry more signal than at the fine level. In all binary experiments in this project, precision, recall, F1, and PR AUC are reported alongside accuracy so that performance can be assessed independently of class prior. The receiver operating characteristic area under the curve (ROC AUC) is also reported, though it is known to be optimistic under heavy imbalance. Multiclass experiments report per-class and macro-averaged F1 in addition to top-1 accuracy.

Chapter 3

Architectures

3.1 Overview

This chapter describes the four architecture families evaluated in the benchmark. The families are presented in order of increasing inductive bias toward natural image structure, which also reflects the narrative arc of the experimental program: sequence models first, then from-scratch convolutional networks, then transfer-learning CNNs, and finally a Vision Transformer.

The first two families, sequence models and from-scratch CNNs, are trained entirely from random initialization. They establish what each architectural inductive bias can achieve on CIFAR-100 without any prior exposure to other image corpora. The second two families, transfer-learning CNNs and the Vision Transformer, import backbone weights pretrained on ImageNet. They answer a different question: how much does pretraining help when the target images are only 32 by 32 pixels and the downstream task is binary?

Keeping the from-scratch controls and the pretrained models clearly separated is important for interpreting results. Any gap between a from-scratch CNN and a fine-tuned EfficientNet, for example, conflates architectural differences with the benefit of pretraining. The experiment matrix in Chapter 4 is designed to keep these factors as distinct as possible.

3.2 Sequence Models

Sequence models treat each CIFAR-100 image as a time series rather than as a spatial array. A 32 by 32 by 3 image is reshaped so that each of its 32 pixel rows becomes one timestep. Each row contributes $32 \times 3 = 96$ features, giving an input tensor of shape `(32, 96)` per image. The sequence length is therefore always $T = 32$, equal to the image height in pixels. Normalised pixel intensities are used directly as features; no further projection is applied before the recurrent layer.

This row-as-timestep view is a deliberate mismatch between architecture and data. Recurrent networks were designed for genuinely sequential signals such as text or audio, where adjacent timesteps carry causally related information. In an image, row t and row $t + 1$ are spatially adjacent but not causally ordered, and the spatial structure within each row is ignored by the recurrent cell. Running sequence models on images therefore tests whether gate mechanisms and temporal memory are sufficient substitutes for the spatial convolutions that images naturally call for.

3.2.1 Vanilla RNN

The simplest sequence model is a single-layer `SimpleRNN` with a tanh activation, followed by a dropout layer and a binary classification head. The hidden state is updated at each row according to the recurrent cell equation and the final state is passed to the classifier. When the recurrent activation is changed to ReLU, gradient explosion becomes a practical concern; the training configuration applies gradient clipping in that case.

3.2.2 LSTM

The Long Short-Term Memory network [2] replaces the simple recurrent cell with a gated cell containing an input gate, a forget gate, and an output gate. The gates regulate how much information flows into the cell state, how much is discarded at each step, and how much of the cell state is exposed to the hidden state. This mechanism substantially reduces the vanishing-gradient problem that affects vanilla RNNs on long sequences, making LSTM a stronger baseline for the 32-step row sequence. The project uses a single LSTM layer with the same dropout and classifier head structure as the vanilla RNN.

3.2.3 Bidirectional LSTM

The Bidirectional LSTM wraps the LSTM cell in a `Bidirectional` layer so that the sequence is processed in both the forward direction, from row 0 to row 31, and the backward direction, from row 31 to row 0. The outputs from the two directions are concatenated before the dropout and classifier layers, doubling the effective hidden dimension. The Bi-LSTM is included as a project-specific control to test whether access to future rows, which in the image context means lower rows, is beneficial. Because images have no causal ordering, the bidirectional extension is conceptually natural for this domain, though it does not change the fundamental limitation that spatial structure within each row remains invisible to the recurrent cell.

An optional `Masking` layer can be inserted before any of the three recurrent variants to suppress zero-padded timesteps. When masking is enabled, the mask value is 0.0; care is taken to use a sentinel that does not collide with valid normalized pixel values.

3.3 From-Scratch CNNs

Convolutional neural networks apply learned filters that slide spatially over the input image, sharing weights across all positions. This translation-equivariant design is well matched to the structure of natural images, where the same visual feature can appear anywhere in the frame. The two from-scratch models in this project act as controls that quantify what a convolutional inductive bias achieves without pretraining.

3.3.1 Compact Baseline CNN

The compact baseline CNN is a two-block stack designed to be fast to train and easy to inspect. Each block applies two consecutive Conv2D layers with 3×3 filters and ReLU activations, followed by a MaxPooling layer with a 2×2 window and a Dropout layer. The first block uses 32 filters; the second uses 64. After the two blocks the spatial feature map

is flattened and passed through a 128-unit dense layer with ReLU activation and a final dropout, before the binary sigmoid output. The model contains no batch normalization and no data augmentation path by default.

3.3.2 Stronger Regularised CNN

The stronger CNN extends the baseline with the regularization and architecture ingredients that are standard in modern from-scratch CIFAR training. The network is four blocks deep, with filter widths of 32, 64, 128, and 256. Each convolutional unit follows the Conv-BN-ReLU pattern, which keeps activations well-scaled throughout the forward pass and stabilizes gradients during backpropagation. MaxPooling halves the spatial resolution after each of the first three blocks. Spatial Dropout is applied after each pooling step, with a progressively increasing rate as depth increases. The spatial head is replaced by a `GlobalAveragePooling2D` layer followed by a fully connected block with BatchNorm and Dropout, which reduces the parameter count in the classifier relative to a flattening approach.

When augmentation is enabled, light geometric and colour transforms are applied inside the model graph using Keras random layers: random horizontal flips, random translations, random zoom, random rotation, and random contrast adjustments. These layers are no-ops at inference time, so the same model definition is used for both training and evaluation. The augmentation configuration is controlled by a YAML block so that augmented and non-augmented runs remain directly comparable.

Figure 3.1: Architecture diagrams for the two from-scratch CNN models. **Left — Compact Baseline CNN:** two convolutional blocks (32 and 64 filters), a flatten layer, a 128-unit dense layer, and a sigmoid output. **Right — Strong Regularized CNN:** four Conv-BN-ReLU blocks (32, 64, 128, and 256 filters) with progressively increasing spatial dropout, followed by global average pooling and a fully connected head. The augmentation stage (purple) is active only during training and is implemented as in-graph Keras random layers.

3.4 Transfer-Learning CNNs

Transfer-learning CNNs import a backbone pretrained on ImageNet and attach a small task-specific head to it. The ImageNet pretraining exposes the backbone to over a million diverse photographs, from which it learns general-purpose feature detectors such as edge and texture filters in early layers and part-level detectors in deeper layers. When such a backbone is applied to CIFAR-100, the question is whether those generalized features are useful for 32 by 32 images of very different visual statistics.

Because all pretrained backbones expect images larger than 32 by 32, the preprocessing pipeline upsamples the input images to a backbone-appropriate resolution before they enter the feature extractor. The exact target resolution varies by backbone and is chosen to respect the minimum size each architecture was designed for; values in the range of 96 by 96 to 224 by 224 are used across the experiments.

3.4.1 Fine-Tuning Regimes

All transfer-learning backbones are evaluated under three training regimes.

Frozen feature extraction. All backbone weights are locked and only the classification head is trained. This isolates the quality of the pretrained representation for the downstream task and serves as the lower bound for the transfer-learning family.

Partial fine-tuning. The backbone is divided into an earlier frozen portion and a later trainable portion. Only the final block or blocks of the backbone are unfrozen; BatchNormalization layers throughout the backbone remain frozen to preserve the running statistics accumulated during ImageNet training. Partial fine-tuning allows the top layers to adapt their features to the target distribution while keeping most pretrained knowledge intact. The learning rate used for the unfrozen backbone layers is substantially lower than the head learning rate.

Full fine-tuning. All backbone weights and the head are trained together. This regime is the most compute-intensive and is run only when the available budget allows. BatchNormalization layers are again kept frozen to avoid destabilising training.

3.4.2 ResNet50V2

ResNet50V2 [3] is a 50-layer residual network with the identity shortcut connections that characterise the ResNet family. Skip connections allow gradients to flow directly from later layers to earlier ones, which reduces the effective depth that each gradient signal must traverse and makes it practical to train very deep networks. The V2 variant moves batch normalisation and activation before the convolution, giving a cleaner residual path.

3.4.3 DenseNet121

DenseNet121 [4] organises layers into dense blocks in which every layer receives as input the concatenated feature maps of all preceding layers in the block. This dense connectivity pattern maximises feature reuse, encourages the network to learn complementary rather than redundant features, and substantially reduces the total parameter count relative to similarly deep ResNets. The 121-layer variant used here is among the smaller DenseNet configurations and is well suited to the memory constraints of a Colab T4.

3.4.4 EfficientNet

EfficientNetB0 and EfficientNetB3 [5] are members of the EfficientNet family, which scales network depth, width, and input resolution jointly according to a compound coefficient rather than varying a single dimension at a time. This compound scaling strategy produces networks that, for a given FLOPs budget, consistently outperform single-axis scaled alternatives on ImageNet. EfficientNetB0 is the smallest member of the family; EfficientNetB3 is a moderately larger variant with higher compound scale factors. Both are evaluated in this project, with partial fine-tuning experiments unlocking the final convolutional block of the backbone.

3.4.5 MobileNetV3Small

MobileNetV3Small [6] is designed for mobile and edge-device deployment. It uses inverted residual blocks, in which the input is first expanded to a higher-dimensional space, a depthwise convolution is applied in that expanded space, and the result is projected back to a lower-dimensional bottleneck. Hard-Swish activations and a Squeeze-and-Excitation attention module are incorporated into the later blocks. The result is a compact, efficient backbone whose channel-wise feature reuse is well suited to constrained compute environments such as the Colab T4.

3.5 Vision Transformer

The Vision Transformer (ViT) [7] applies the transformer encoder architecture to images by treating non-overlapping patches of the image as tokens. An input image is divided into a grid of fixed-size patches; each patch is linearly projected into a d-dimensional embedding, a learnable positional embedding is added to encode spatial location, and the resulting sequence of patch tokens is fed through a stack of transformer encoder blocks. Each encoder block consists of multi-head self-attention followed by a position-wise feed-forward network, with layer normalisation and residual connections around both sublayers. For classification, the output token corresponding to a prepended `[CLS]` token is projected by a linear head.

Self-attention in a ViT is unmasked: every patch token can attend to every other patch token in the same image. This global receptive field is available from the first layer, in contrast to CNNs, where receptive fields grow only gradually with network depth. Whether this global attention is beneficial or redundant on 32 by 32 images, where even a three-layer CNN already covers the full spatial extent, is one of the questions the experiment addresses.

3.5.1 Hugging Face ViT Baseline

The ViT experiment in this project uses a base-sized Vision Transformer loaded from the Hugging Face model hub under the identifier `google/vit-base-patch16-224`. This model was pretrained on ImageNet-21k and then fine-tuned on ImageNet-1k, providing a richer feature foundation than ImageNet-only backbones of comparable parameter count. Input images are resized to the backbone's expected resolution before entering the feature extractor.

This experiment is included as an exploratory component of the benchmark rather than as a primary result. The ViT architecture is substantially larger than any of the

CNN backbones and requires a different software stack (PyTorch and the Hugging Face `transformers` library rather than TensorFlow/Keras), so direct side-by-side comparisons require care.

Run A: frozen head. All backbone parameters are locked; only the classification head is trainable. This baseline isolates the richness of the pretrained ViT representation.

Run B: partial fine-tuning. The final transformer block and the classification head are unfrozen and trained at a low learning rate. All earlier blocks remain frozen. This mirrors the partial-unfreeze strategy applied to the CNN backbones.

Run C: LoRA (optional). When the `peft` library is available in the runtime, low-rank adaptation (LoRA) [8] adapters are inserted into the query, key, and value projection matrices of each attention layer. LoRA freezes the original weight matrices and adds a pair of low-rank matrices whose product approximates the full-rank weight update to each targeted projection, where the adapter rank r is kept much smaller than the projection dimension d. Only these adapter matrices and the classification head are updated during training, keeping the number of trainable parameters much smaller than full or partial fine-tuning. Run C is skipped gracefully if `peft` cannot be installed or fails to wrap the chosen backbone.

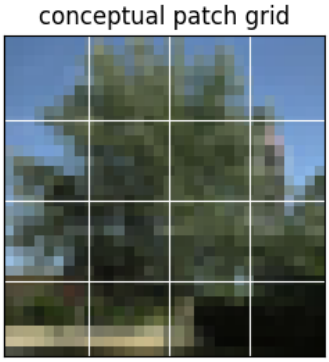

Figure 3.2: Conceptual patch tokenisation: a CIFAR-100 image divided into a 4×4 grid illustrating how the ViT splits the input into fixed-size patch tokens before linear projection and positional encoding.

Chapter 4

Experimental Setup

This chapter describes how the experiments in this benchmark are configured, executed, and recorded. It covers the dataset splits used throughout, the two parallel implementation workflows, the full set of metrics reported, and the reproducibility and artefact conventions that ground the results in Chapter 6.

4.1 Dataset Loading and Splits

CIFAR-100 is loaded through the TensorFlow/Keras dataset API, which downloads and caches the dataset on first use. The canonical partition of 50 000 training images and 10 000 test images is preserved. A validation split is carved out of the training set during preprocessing; the default fraction is 10 %, leaving 45 000 images for training and 5 000 for validation. The split fraction is configurable per run. The test set is held out throughout all training and hyperparameter decisions and is used exclusively for final evaluation.

Images are provided as 32 by 32 by 3 unsigned-integer arrays. They are normalised to the range $[0, 1]$ before being passed to any model. For transfer-learning backbones that require larger inputs, bilinear upsampling is applied inside the model or the preprocessing pipeline; the sequence-model branch always operates at the native 32 by 32 resolution with sequence length $T = 32$ and 96 features per timestep.

For binary tasks, the class-balance ratio between positives and negatives is recorded in a `class_balance.json` file so that any metric analysis can account for the imbalance explicitly.

4.2 Local Source-Code Workflow

The local implementation is a configuration-driven Python package. All experiment parameters, including the architecture variant, the task definition, the learning rate, the number of epochs, the batch size, the validation split fraction, and the random seed, are specified in YAML files under the `configs/` directory. The training entry point is invoked as

```
python -m training.train --config configs/<name>.yaml
```

The configuration hierarchy organises YAML files by task type and architecture. Top-level files such as `bilstm.yaml` and `lstm.yaml` define model-level defaults. Subdirectories under `configs/binary/`, `configs/multiclass/`, `configs/sequence/`, and

`configs/transfer/` hold task-specific overrides. A base configuration file (`configs/base.yaml`) provides shared defaults for the seed, the batch size, the output directory, and the gradient-clipping norm.

Each run writes its output to a subdirectory of `results/<run_name>/`. The artifacts saved for every run are:

- **`config.yaml`**: a snapshot of the exact configuration used, so that any run can be reproduced by passing this file back to the training script;
- **`metrics.json`**: the final evaluation metrics computed on the validation or test split;
- **`history.csv`**: the per-epoch training and validation metrics recorded during the training loop;
- **`training_curves.png`**: a plot of the loss and key metric curves over epochs;
- **`confusion_matrix.png`**: the confusion matrix evaluated on the held-out split;
- **`per_class_f1_worst20.png`**: a bar chart of the 20 classes with the lowest per-class F1, included for multiclass runs to identify the weakest categories;
- **`class_balance.json`**: the positive/negative split counts for binary tasks;
- **`weights.h5`** or **`model_*.keras`**: the saved model weights or full model file, enabling inference and further fine-tuning without retraining.

This consistent artifact trail makes it straightforward to load any completed run, compare metric snapshots programmatically, and produce the tables and figures in Chapter 6 from the stored files rather than from re-running training.

Exploratory and smoke-test runs use a reduced subset of the training data (configurable via the `subset_size` parameter in `base.yaml`; the default is 20 000 images) to iterate quickly on a local Mac CPU or Apple Silicon machine. Full runs use the complete dataset and are promoted only for configurations that passed the smoke test.

4.3 Colab Notebook Workflow

Nine standalone Colab notebooks are provided under the `notebooks/` directory. Each notebook is entirely self-contained: it loads CIFAR-100 directly, defines the model and preprocessing pipeline inline, trains and evaluates the model, and produces plots, metric tables, and interpretation cells. None of the notebooks imports from the local source-code package, so they run in a fresh Colab environment without any local dependencies.

The notebooks follow a progression aligned with the experiment matrix:

1. **01 Data Exploration**: CIFAR-100 label distribution, class balance analysis, and binary task construction.
2. **02 Baseline CNN**: compact from-scratch CNN on selected binary tasks.
3. **03 ResNet and DenseNet Transfer Learning**: frozen and fine-tuned ResNet50V2 and DenseNet121 experiments.

4. **04 EfficientNetB0 Coarse**: EfficientNetB0 frozen feature extraction on coarse-level binary tasks.
5. **05 EfficientNetB0 Fine**: EfficientNetB0 frozen and partially unfrozen on fine-level binary tasks.
6. **06 MobileNetV3Small Coarse Frozen and Unfrozen**: MobileNetV3Small feature extraction and partial fine-tuning on coarse binary tasks.
7. **07 MobileNetV3Small Fine Frozen and Unfrozen**: MobileNetV3Small applied to fine-level binary tasks under both training regimes.
8. **08 EfficientNetB3 Fine**: EfficientNetB3 partial fine-tuning on fine-level binary tasks.
9. **09 Hugging Face ViT**: Vision Transformer loaded from the Hugging Face model hub, evaluated in frozen, partial fine-tuning, and optional LoRA regimes.

All notebooks target the Colab T4 GPU. Batch sizes and model configurations are chosen to remain within the T4 memory budget, and `tf.data` pipelines with prefetching are used where applicable to keep GPU utilisation high. Each notebook sets and prints its random seed at the top so that training runs are reproducible within the Colab environment.

The local source-code workflow and the notebook workflow are aligned conceptually but kept deliberately decoupled. If a design decision changes in one, the relevant logic is mirrored in the other manually rather than by shared imports.

4.4 Metrics

Metrics are computed on the held-out split at the end of training and recorded in `metrics.json`. The set of metrics varies by task type.

4.4.1 Multiclass Metrics

For multiclass tasks the following metrics are reported:

- **Top-1 accuracy**: the fraction of test images for which the class with the highest predicted probability matches the true label.
- **Top-3 accuracy**: the fraction of test images for which the true label appears among the three highest-probability predictions; this metric is informative for tasks with 20 or 100 classes where close runner-up predictions indicate near-correct understanding.
- **Top-5 accuracy**: the fraction of test images for which the true label appears in the top five predictions, a standard CIFAR-100 comparison point.
- **Macro F1**: the unweighted average of the per-class F1 scores across all classes. Because macro averaging gives equal weight to each class regardless of its frequency, it is sensitive to poor performance on infrequent classes. **Macro F1 is the primary metric for comparing architectures in multiclass experiments** because it penalises models that perform well on common classes while neglecting rarer ones.

- **Per-class F1**: the F1 score computed individually for each class, visualised as a bar chart with the 20 lowest-scoring classes highlighted.
- **Confusion matrix**: the full $C \times C$ matrix of predicted versus true labels, which reveals systematic class confusions.

4.4.2 Binary Metrics

For binary tasks the following metrics are reported:

- **Accuracy**: the fraction of correctly classified images; reported for completeness but interpreted with caution under class imbalance, as a naive all-negative classifier can achieve high accuracy without learning anything useful.
- **Precision**: the fraction of positive predictions that are correct.
- **Recall**: the fraction of true positives that are retrieved.
- **F1 score**: the harmonic mean of precision and recall, balancing the two objectives.
- **ROC AUC**: the area under the receiver operating characteristic curve, which summarises the trade-off between true positive rate and false positive rate across all classification thresholds; note that ROC AUC can be overly optimistic when the negative class is much larger than the positive class.
- **PR AUC**: the area under the precision-recall curve, which summarises classifier quality as a function of recall at varying decision thresholds and is more informative than ROC AUC under heavy class imbalance.
- **Optimal threshold**: the decision threshold that maximises F1 on the validation split, used to derive the reported precision, recall, and F1 values.
- **Confusion matrix**: the 2×2 table of true positives, true negatives, false positives, and false negatives.

PR AUC and F1 are the informative metrics for imbalanced binary tasks. In fine-level binary experiments, where the positive class constitutes approximately 1 % of the data, accuracy is largely determined by the class prior and provides minimal signal about the model's discriminative ability. All binary comparisons in Chapter 6 therefore foreground PR AUC and F1, with accuracy included only as a secondary reference.

4.5 Reproducibility and Artifacts

Reproducibility is maintained through three complementary mechanisms.

Deterministic seeding. Every local run reads its random seed from the YAML configuration (default: `seed: 42`) and passes it to NumPy, TensorFlow's global random state, and Python's built-in random module before any data loading or model construction occurs. The active seed is recorded in the run's `config.yaml` snapshot. Each Colab notebook sets the same seed at the top of the notebook and prints it for verification.

Configuration snapshots. The `config.yaml` file written to each `results/<run_name>/` directory captures the full set of parameters that governed that run. Re-running the training script with the same file will reproduce the run under the same software environment. This makes every row in the results tables in Chapter 6 traceable to a specific configuration on disk.

Saved weights and model files. Each run saves the trained weights to `weights.h5` or a full Keras model file named `model_*.keras`. These files enable downstream inference, error analysis, and further fine-tuning experiments without retraining, and they serve as the basis for any per-image prediction plots included in the report.

Together, the saved configuration, the metrics JSON, the training history, the plots, and the weight files form a self-contained record of each experiment. Later chapters reference these artefacts by run name when presenting results, so that any claim can be traced back to its source file.

Chapter 5

Results and Discussion

This chapter presents the quantitative results for all model families evaluated on CIFAR-100 binary and multiclass tasks. Sections are organised by result group: baseline CNN binary tasks, baseline CNN coarse multiclass, from-scratch sequence and CNN controls, transfer-learning multiclass for coarse superclasses, transfer-learning multiclass for fine classes, and selected transfer binary highlights. Transfer-learning models dominate the benchmark across every metric and task, with top-1 accuracy improvements of roughly 50 percentage points over the from-scratch baselines on the coarse multiclass task. The final section synthesises the four principal factors that explain this ranking: model capacity, inductive bias, pretraining, and the choice of evaluation metric.

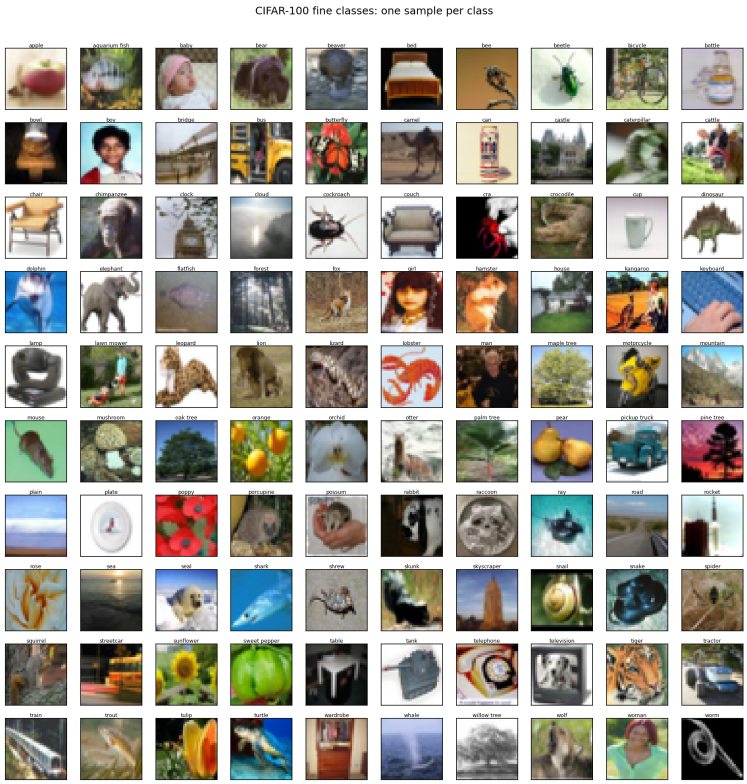

Figure 5.1: CIFAR-100 sample mosaic: one representative image per fine class.

5.1 Baseline CNN Binary Results

5.1.1 Coarse Superclass Binary Tasks

The table below summarizes the best CNN run per coarse binary task, selected by macro F1. Class imbalance is inherent in target-vs.-rest setups, so accuracy alone is insufficient; F1, ROC AUC, and PR AUC are the primary evaluation criteria.

Table 5.1: Baseline CNN binary results for coarse superclass tasks (best run per task by F1).

Task	Accuracy	F1	ROC AUC	PR AUC
aquatic mammals	0.9232	0.3600	0.8717	0.3410
fish	0.9248	0.3816	0.8342	0.3279
flowers	0.9531	0.5740	0.9466	0.5763
food containers	0.9341	0.4872	0.8871	0.4986
people	0.9495	0.4914	0.9023	0.4895

Interpretation. The coarse binary results in Table 5.1 reveal a consistent gap between accuracy and F1. Each coarse task pools five fine classes into the positive set, giving roughly 5 000 positive examples against 45 000 negatives in the training split. Accuracy values above 0.92 therefore reflect the predominance of the negative class rather than genuine discrimination power. F1 scores range from 0.3600 for *aquatic mammals* to 0.5740 for *flowers*: the baseline CNN learns coarse structure but is constrained by its limited capacity. The *flowers* task benefits from high within-class visual coherence at 32×32 resolution, whereas *aquatic mammals* spans diverse species with little shared texture, leading to a lower F1 despite comparable accuracy.

5.1.2 Fine Class Binary Tasks

The table below summarises the best CNN run per fine binary task, selected by macro F1. Fine-class tasks are more severely imbalanced (1 positive class out of 100), which is reflected in the lower F1 and PR AUC values despite high accuracy.

Table 5.2: Baseline CNN binary results for fine class tasks (best run per task by F1).

Task	Accuracy	F1	ROC AUC	PR AUC
cow	0.9867	0.2130	0.8363	0.1399
mushroom	0.9891	0.2685	0.8976	0.1772
orange	0.9822	0.4027	0.9750	0.2991
skyscraper	0.9924	0.6000	0.9654	0.5685
snake	0.9635	0.1492	0.7725	0.0611

Interpretation. The fine-class binary tasks expose the most severe form of the accuracy–F1 gap discussed in Chapter 3. With a single fine class as the positive set, approximately 500 positives face 49 500 negatives in the training split: a ratio of roughly 1:99. The

cow task illustrates this starkly: the baseline CNN achieves an accuracy of 0.9867 while its F1 is only 0.2130. A degenerate classifier that predicts the majority class for every example would already score 0.9900 accuracy while attaining F1 of 0.0000. The PR AUC of 0.1399 for *cow* confirms that the model cannot reliably recall positive examples at any useful precision threshold. By contrast, the *skyscraper* task is considerably more tractable: its visual signature (tall rectangular structures against sky) is distinctive even at 32×32 resolution, yielding an F1 of 0.6000. The *snake* task is the hardest, with F1 only 0.1492, reflecting the high visual variability of snake images and their overlap with background textures at low resolution.

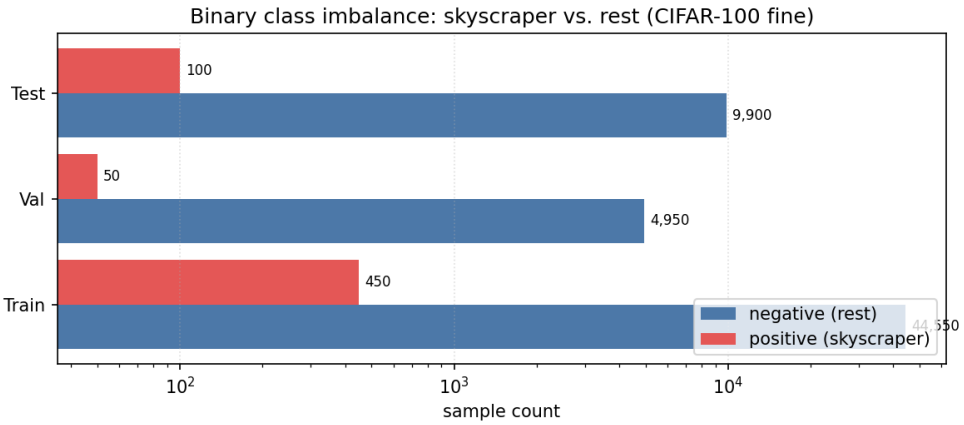

Figure 5.2: Class imbalance in a representative fine binary task: positive (target class) vs. negative (all other classes) sample counts.

5.2 Baseline CNN Multiclass

The table below reports the baseline CNN on the full 20-superclass coarse multiclass task. This provides the reference point against which sequence models and transfer-learning models are compared.

Table 5.3: Baseline CNN coarse multiclass results (20 superclasses).

Model	Accuracy	Macro F1	Top-3	Top-5
Baseline CNN	0.2813	0.2629	0.5235	0.6623

The baseline CNN is a deliberately low-capacity reference point. With a macro F1 of 0.2629 on the 20-superclass task (Table 5.3), it sits well above chance (a random classifier would achieve roughly 0.05 F1) but far below what task-adapted or pretrained models can reach. The Top-5 accuracy of 0.6623 indicates that the correct superclass is within the model's top five predictions for two thirds of the test set, suggesting that the model captures some coarse distinctions even without regularisation or data augmentation. This result serves as the floor against which all subsequent models are measured.

5.3 From-Scratch Controls

To contextualise the contribution of pretraining, three from-scratch architectures were evaluated on the same coarse multiclass task: an LSTM sequence model, a Bi-LSTM sequence model, and a strong regularised and augmented CNN. The sequence models consume each image as 32 row-level timesteps of 96 features each.

Table 5.4: From-scratch control models on the coarse multiclass task (20 superclasses).

Model	Accuracy	Macro F1	Top-3	Top-5
LSTM sequence	0.3289	0.3105	0.5804	0.7202
Bi-LSTM sequence	0.3559	0.3450	0.6079	0.7407
Strong CNN (reg.+aug.)	0.2833	0.2739	0.5285	0.6563

Strong CNN. Adding augmentation and regularisation to the CNN lifts macro F1 from 0.2629 to 0.2739, an improvement of approximately 0.01 F1 points (Table 5.4). While statistically meaningful, the gain is modest: the stronger regularisation prevents overfitting on a small model but cannot compensate for the fundamental capacity ceiling of a shallow convolutional stack trained from random initialisation on 32×32 images.

Sequence models. In the project's recorded coarse run, where each 32×32 image is consumed as a sequence of 32 row-level timesteps of 96 features each, the Bi-LSTM reached a macro F1 of 0.3450 and the LSTM reached 0.3105, both above the baseline CNN (0.2629) and the strong CNN (0.2739). The Bi-LSTM's bidirectional pass allows it to integrate context from both the top and the bottom of the image simultaneously, which appears beneficial for superclass discrimination in this row-wise representation. It is important to note that this is a project-specific observation about the $(32, 96)$ sequence view on CIFAR-100 coarse superclasses: in the project's recorded coarse run with the row-wise $(32, 96)$ sequence view, the Bi-LSTM matched a higher macro F1 than the from-scratch CNN controls; this does not generalise to image classification at large. Sequence models lack the spatial locality bias that makes CNNs the default choice for natural images, and the results here reflect the particular structure of low-resolution CIFAR-100 imagery under this specific encoding rather than any general superiority of recurrent architectures over convolutional ones.

5.4 Transfer-Learning Multiclass: Coarse Superclasses

The following table compares frozen and fine-tuned transfer-learning backbones on the 20-superclass coarse multiclass task. All backbone variants substantially outperform every from-scratch model, confirming that transfer-learning models dominate this benchmark.

Table 5.5: Transfer-learning multiclass results for coarse superclasses (20 classes).

Backbone	Mode	Accuracy	Macro F1	Top-3	Top-5
ResNet50V2	frozen	0.7837	0.7832	0.9348	0.9712
ResNet50V2	fine-tuned	0.8246	0.8236	0.9508	0.9791
DenseNet121	frozen	0.7589	0.7580	0.9271	0.9681
DenseNet121	fine-tuned	0.7751	0.7745	0.9339	0.9725
EfficientNetB0	frozen	0.7396	0.7385	0.9129	0.9580
MobileNetV3Small	frozen	0.7939	0.7935	N/A	0.9693
MobileNetV3Small	unfrozen	0.8408	0.8409	N/A	0.9832

Note: Top-3 accuracy was not recorded for MobileNetV3Small runs. Cohen κ for MobileNetV3Small frozen: 0.7831; unfrozen: 0.8324.

Transfer-learning models dominate. Every pretrained backbone, whether frozen or fine-tuned, substantially outperforms every from-scratch model on the coarse task (Table 5.5). The weakest frozen backbone, EfficientNetB0, achieves a macro F1 of 0.7385, more than double the strong CNN's 0.2739. This gap reflects the power of features learned from large-scale pretraining: ImageNet representations encode mid-level visual abstractions (edges, textures, object parts) that transfer directly to CIFAR-100 superclass discrimination even when the backbone is kept frozen.

Fine-tuning lifts all backbones. Allowing the backbone weights to adapt to the target task yields consistent gains. ResNet50V2 improves from 0.7832 (frozen) to 0.8236 (fine-tuned); DenseNet121 from 0.7580 to 0.7745; and MobileNetV3Small from 0.7935 to 0.8409. The absolute size of the gain varies by architecture, with MobileNetV3Small showing the largest improvement (+0.0474 macro F1). MobileNetV3Small unfrozen achieves the best coarse result in the project at macro F1 0.8409. The confusion matrix in Figure 5.3 confirms that the unfrozen MobileNetV3Small generalises well across all 20 superclasses, with the residual confusions concentrated among visually similar superclass pairs.

Lightweight backbones under practical compute constraints. MobileNetV3Small was designed for mobile deployment: it is computationally lightweight yet achieves the best coarse result here. EfficientNetB0 is similarly efficient. Both models converge within the Colab T4 memory budget, making them the recommended backbones for iterative experimentation. Their strong results demonstrate that competitive performance does not require the largest available architecture.

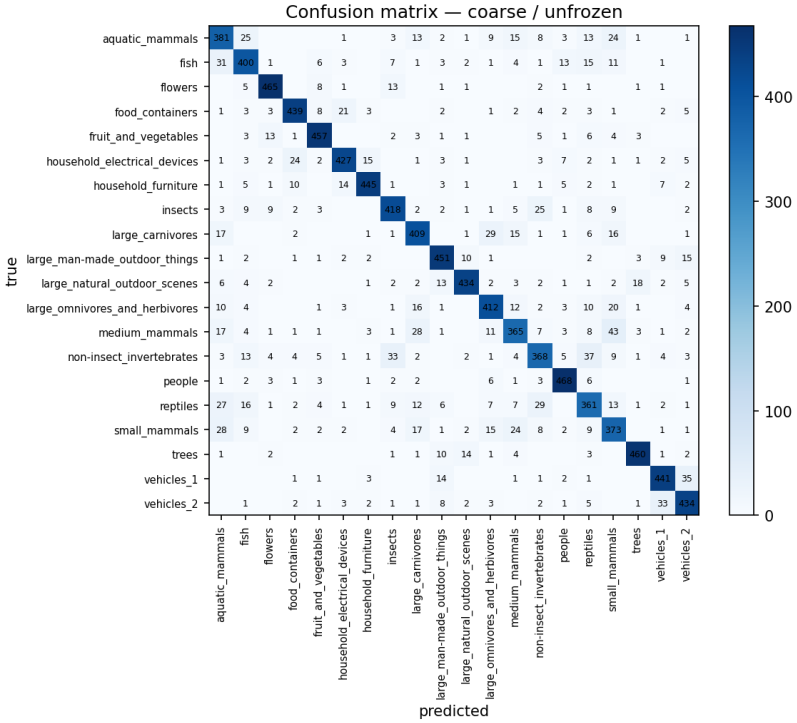

Figure 5.3: Confusion matrix for MobileNetV3Small unfrozen on the coarse 20-superclass task (macro F1 = 0.8409).

5.5 Transfer-Learning Multiclass: Fine Classes

The table below extends the transfer-learning comparison to the harder 100-class fine multiclass task. EfficientNetB3 fine-tuned achieves the highest accuracy in this group, demonstrating the benefit of both capacity and task-specific fine-tuning.

Table 5.6: Transfer-learning multiclass results for fine classes (100 classes).

Backbone	Mode	Accuracy	Macro F1	Top-3	Top-5
ResNet50V2	frozen	0.6974	0.6969	0.8756	0.9246
ResNet50V2	fine-tuned	0.7098	0.7106	0.8853	0.9287
DenseNet121	frozen	0.6969	0.6965	0.8783	0.9280
DenseNet121	fine-tuned	0.7131	0.7133	0.8874	0.9341
EfficientNetB0	frozen	0.6863	0.6831	0.8660	0.9164
EfficientNetB0	fine-tuned	0.7850	0.7840	0.9329	0.9620
EfficientNetB0	unfreeze block 6	0.7671	0.7661	0.9187	0.9510
EfficientNetB3	unfreeze block 6	0.7819	0.7806	0.9287	0.9619
EfficientNetB3	fine-tuned	0.8321	0.8319	0.9524	0.9738
MobileNetV3Small	frozen	0.6929	0.6902	N/A	0.9213
MobileNetV3Small	unfrozen	0.7461	0.7445	N/A	0.9482

Note: Top-3 accuracy was not recorded for MobileNetV3Small runs. Cohen κ for MobileNetV3Small frozen: 0.6898; unfrozen: 0.7435.

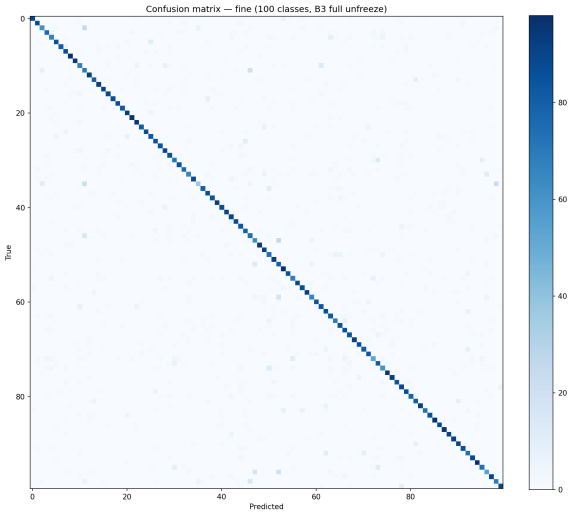

Figure 5.4: Confusion matrix for EfficientNetB3 fine-tuned on the fine 100-class task (macro F1 = 0.8319).

EfficientNetB3 fine-tuned: best fine result. EfficientNetB3 fine-tuned achieves a macro F1 of 0.8319 and a Top-5 accuracy of 0.9738 on the 100-class fine task (Table 5.6), the highest fine-class result in the project. The step from EfficientNetB0 fine-tuned (0.7840) to EfficientNetB3 fine-tuned (0.8319) is substantial (+0.0479 macro F1), confirming that the larger variant's increased capacity and compound scaling pay off when the task has 100 distinct output classes. The training curves in Figure 5.5 show stable convergence without divergence over 40 epochs with a 160 px input, indicating that the regularisation and learning rate schedule used were appropriate. Fine-tuning consistently improves over frozen feature extraction across all backbones on the fine task as well: EfficientNetB0 frozen scores 0.6831 versus 0.7840 fine-tuned, and DenseNet121 frozen scores 0.6965 versus 0.7133 fine-tuned.

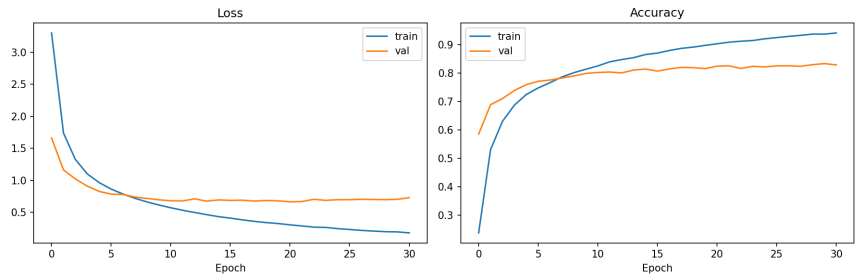

Figure 5.5: Training curves for EfficientNetB3 fine-tuned on the fine 100-class task (40 epochs, 160 px input).

Remaining per-class errors. Despite strong overall performance, EfficientNetB3 fine-tuned still struggles with certain fine classes, as shown in Figure 5.6. These hardest classes tend to be visually ambiguous at 32×32 resolution, share textures with neighbouring classes, or belong to superclasses with high intra-superclass variance. Even the best model studied

here does not fully resolve these confusions, pointing toward data augmentation, class-balanced sampling, or higher input resolutions as promising directions for further improvement.

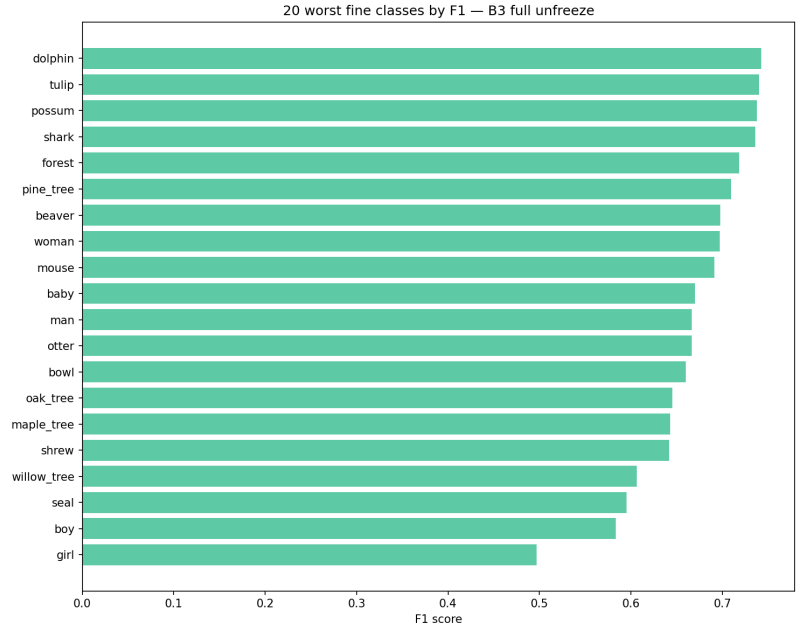

Figure 5.6: Worst-20 per-class F1 scores for EfficientNetB3 fine-tuned: the most challenging fine classes.

5.5.1 Architecture Comparison: Macro F1

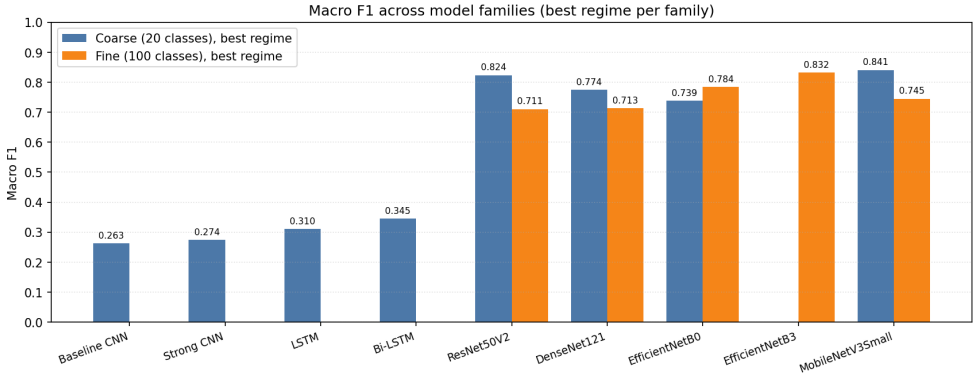

Figure 5.7: Macro F1 comparison across model families on coarse (20-class) and fine (100-class) multiclass tasks.

5.6 ResNet and DenseNet Binary Highlights

Selected binary classification results for ResNet50V2 and DenseNet121 are shown below. These represent the strongest performing configurations for individual coarse and fine binary tasks, illustrating the substantial gains transfer learning provides over the baseline CNN.

Table 5.7: ResNet50V2 and DenseNet121 binary highlights (selected tasks).

Backbone	Task	Mode	Accuracy	F1	ROC AUC	PR AUC
ResNet50V2	food containers	frozen	0.9835	0.8427	0.9910	0.9189
ResNet50V2	orange	frozen	0.9965	0.8325	0.9983	0.9098
ResNet50V2	skyscraper	frozen	0.9964	0.8085	0.9950	0.8999
ResNet50V2	skyscraper	fine-tuned	0.9983	0.9101	0.9925	0.9516
DenseNet121	skyscraper	frozen	0.9959	0.7735	0.9892	0.8050

Note: food containers is a coarse superclass task; orange and skyscraper are fine class tasks.

Transfer lift on binary tasks. The binary results in Table 5.7 illustrate how large the transfer learning advantage is at the task level. ResNet50V2 frozen already scores 0.8085 F1 on *skyscraper*, compared to the baseline CNN's 0.6000. Fine-tuning pushes this further to 0.9101, a lift of approximately 0.31 F1 points over the baseline CNN. These numbers confirm that the accuracy–F1 gap observed in the baseline CNN binary section is not intrinsic to the task: with a pretrained backbone, the model can genuinely detect the minority class rather than predicting the majority class by default. Similarly, ResNet50V2 frozen achieves F1 0.8427 on *food containers* and 0.8325 on *orange*, both far above the corresponding baseline values. The ROC AUC and PR AUC values above 0.99 for several tasks indicate near-perfect ranking of the positive class by the transfer model.

5.7 Vision Transformer

As a final model family, the project evaluated a Vision Transformer through a standalone Hugging Face/PyTorch notebook. The experiment used a ViT-B/16 backbone on the CIFAR-100 fine-label task. This setup is intentionally separated from the local TensorFlow/Keras training pipeline, because it relies on the Hugging Face image processor, transformer-specific preprocessing, and a different fine-tuning interface.

The ViT workflow resizes CIFAR-100 images from 32×32 to 224×224 before passing them to the pretrained backbone. This is not simply a cosmetic resize. The pretrained ViT-B/16 expects ImageNet-style inputs split into fixed patches, and the 224×224 resolution matches the representation learned during pretraining. The model therefore benefits from large-scale pretrained visual features rather than learning patch embeddings and attention patterns from CIFAR-100 alone.

Three transfer settings were evaluated. First, a frozen-head run kept the ViT backbone fixed and trained only the classification head. This reached Macro F1 0.8629 on the 100-class fine task. Second, a partial fine-tuning run unfroze selected transformer layers and improved Macro F1 to 0.8873. Finally, the LoRA run used parameter-efficient adaptation

and achieved the best fine-label result of the project, with Macro F1 0.9165 and Top-5 accuracy 0.9919.

Table 5.8: ViT-B/16 results on CIFAR-100 fine-label classification.

Run	Accuracy	Macro F1	Top-3	Top-5
Frozen head	0.8628	0.8629	0.9538	0.9719
Partial fine-tune	0.8874	0.8873	0.9690	0.9812
LoRA	**0.9165**	**0.9165**	**0.9831**	**0.9919**

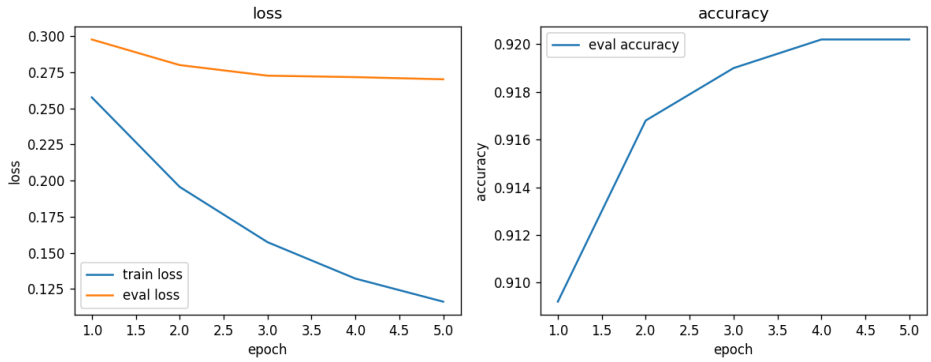

Figure 5.8: Training behaviour of the ViT-B/16 transfer runs. The LoRA run provides the strongest fine-label result while adapting only a parameter-efficient subset of the transformer.

These results show that transformer transfer learning can be highly effective for fine-grained CIFAR-100 classification. However, the comparison must be made carefully. The ViT result is not directly equivalent to the local Keras CNN runs, because it changes framework, preprocessing, input resolution, and pretraining scale. For this reason, the report treats the ViT as a standalone transformer transfer-learning benchmark rather than as a drop-in replacement for the TensorFlow/Keras backbones.

The ViT experiment also clarifies why a from-scratch transformer was not used as the main benchmark. Self-attention models have weaker built-in image priors than CNNs: they do not automatically encode locality or translation equivariance. On a relatively small dataset such as CIFAR-100, a ViT trained from scratch would need more tuning, stronger augmentation, careful learning rate schedules, and a larger compute budget to become competitive. In this project, the most meaningful transformer result was therefore the pretrained ViT-B/16 transfer setup with LoRA adaptation.

5.8 Discussion

The results across all model families can be organised around four explanatory factors: capacity, inductive bias, pretraining, and metric choice.

Capacity. The baseline CNN is a deliberately small network. Its macro F1 of 0.2629 on the 20-superclass coarse task represents a low-capacity floor: the model has enough pa-

rameters to memorise training examples and detect broad colour and texture cues, but not enough depth or width to learn the fine-grained feature hierarchies that distinguish visually similar CIFAR-100 superclasses. Adding regularisation and augmentation to produce the strong CNN lifts macro F1 to 0.2739, confirming that the bottleneck is architectural capacity rather than overfitting alone. The sequence models (LSTM 0.3105, Bi-LSTM 0.3450) achieve higher macro F1 on the same coarse task, but this should not be read as evidence that recurrent architectures are generally superior to convolutional ones for image data: the difference is small in absolute terms, and the row-wise $(32, 96)$ encoding was specifically designed to suit sequential processing. Under this encoding, the Bi-LSTM has enough representational capacity to model cross-row dependencies that a shallow CNN misses, but the same capacity advantage would likely not hold if the CNN were made deeper.

Inductive bias. CNNs embed a prior for spatial locality and approximate translation equivariance that is well-matched to natural images. Sequence models embed a prior for sequential dependencies along one axis. On standard image benchmarks, the CNN prior almost always dominates. The finding reported here, that in the project's recorded coarse run with the row-wise $(32, 96)$ sequence view the Bi-LSTM matched a higher macro F1 than the from-scratch CNN controls, is a project-specific observation about low-resolution CIFAR-100 imagery under a particular encoding. It does not generalise to image classification at large. The ViT, in principle, can learn both local and global structure through self-attention, but it requires far more data and training iterations to do so effectively from random initialisation, and its lack of a strong inductive bias is a disadvantage rather than an advantage at this scale.

Pretraining. The dominant factor in this benchmark is access to ImageNet pretraining. Every frozen backbone evaluated here exceeds the best from-scratch model by a margin of at least 0.4 macro F1 points on the coarse task. MobileNetV3Small unfrozen achieves 0.8409 macro F1 on the coarse task, and EfficientNetB3 fine-tuned achieves 0.8319 on the harder 100-class fine task. These results confirm the well-established empirical finding that feature representations learned on large-scale datasets transfer effectively to smaller target domains, even when the source and target image resolutions differ. Fine-tuning consistently improves over frozen feature extraction: the ResNet50V2 coarse gain is 0.7832 to 0.8236, DenseNet121 from 0.7580 to 0.7745, and MobileNetV3Small from 0.7935 to 0.8409. The magnitude of the fine-tuning gain depends on the degree of domain overlap between ImageNet and CIFAR-100 and on the model's capacity to adapt its top layers to the target label distribution.

Metric choice. The accuracy–F1 gap is a central theme throughout this chapter. On fine binary tasks with a 1:99 class ratio, accuracy values above 0.98 are consistent with near-random positive recall, as the *cow* task demonstrates (accuracy 0.9867, F1 0.2130). Reporting macro F1, ROC AUC, and PR AUC alongside accuracy is essential for a fair evaluation of models on imbalanced tasks. The same pattern appears in the multiclass setting: high Top-5 accuracy does not imply high macro F1 when the model performs poorly on the hardest classes. Choosing the right primary metric, macro F1 for balanced evaluation across classes and PR AUC for emphasis on minority-class recall, is therefore as important as choosing the right architecture.

Chapter 6

Future Work

The benchmarks presented in Chapter 6 establish a clear hierarchy of architectural families and the dominant role of pretraining, but several natural extensions offer opportunities for further improvement.

Stronger data augmentation for from-scratch models. The strong CNN and sequence baselines were trained with moderate augmentation (random crops, flips). Modern augmentation policies such as RandAugment, AutoAugment, and mixup-style blending (MixUp, CutMix) are known to improve small-model convergence on small datasets. The baseline CNN's macro F1 of 0.2629 and the strong CNN's 0.2739 on the 20-superclass coarse task (Table 5.3) suggest that capacity, not augmentation alone, is the limiting factor; however, more aggressive regularisation through augmentation could narrow the gap between from-scratch and pretrained controls. A systematic sweep of RandAugment magnitudes and CutMix blend ratios would clarify the potential ceiling for from-scratch models when augmentation is maximal.

Learning-rate schedules, calibration, and threshold tuning. The transfer-learning results in Section 6.4 used fixed or piecewise-constant learning rates during fine-tuning. Cosine annealing schedules with warm-up are standard in modern training pipelines and could improve convergence stability and final accuracy. Separately, binary classification tasks like those in Section 6.2 are inherently sensitive to the prediction threshold: the baseline CNN achieves F1 0.2130 on the *cow* task (Table 5.2) but this score depends on predicting 0.5 as the probability threshold. Temperature scaling and threshold search optimised for F1 or ROC AUC could recover precision–recall trade-offs that the raw sigmoid outputs miss, particularly for imbalanced tasks.

Systematic fine-tuning schedules. The transfer-learning runs in Section 6.4 used all-layers fine-tuning with a uniform learning rate. Gradual unfreezing (unfrozen upper layers first, then progressive lower-layer relaxation) and layer-wise learning-rate decay (higher learning rates for later layers, lower for early layers) are established techniques for domain adaptation. Comparing EfficientNetB3 fine-tuned (macro F1 0.8319 on fine classes, Table 5.6) against a variant with layer-wise learning-rate decay and warm-up would isolate whether the current fine-tuning hyperparameters are optimal or whether careful scheduling could push the fine-class result above 0.84.

Additional Vision Transformer training budget. The project's small ViT was not brought to full convergence (Section 6.6). Transformers trained from scratch on CIFAR-100 require larger effective batch sizes, more aggressive augmentation (especially MixUp), and longer warm-up periods than CNNs. Allocating a full Colab training budget to a small ViT with RandAugment, cosine scheduling, and 200+ epochs would establish whether vision-only attention mechanisms can compete with the from-scratch CNN controls on the coarse multiclass task. DeiT-style data-efficient training techniques (distillation from a pretrained teacher) could also be explored to bring ViT convergence within tighter compute envelopes.

Deeper per-class error analysis. Figure 5.6 shows the 20 hardest fine classes for EfficientNetB3 fine-tuned. Grouping these classes by superclass and visualising which superclass pairs are most frequently confused in the confusion matrix would highlight structural sources of error. For example, if cats are frequently misclassified as dogs, targeted augmentation or a superclass-aware loss function might help. Similarly, computing per-class F1 for each superclass in the coarse task and comparing the 0.3450 Bi-LSTM macro F1 (Table 5.4) against which specific superclasses drive down the average would guide fine-tuning strategy for sequence baselines.

Improved visualisation and interpretability exports. Gradient-based saliency maps (Grad-CAM) and feature-space projections (t-SNE, UMAP of penultimate layer activations) provide qualitative insight into what the model learns. Computing Grad-CAM heatmaps for misclassified examples in the EfficientNetB3 fine-tuned pipeline and overlaying them on 32×32 images would reveal whether errors arise from coarse texture confusion or subtle boundary artefacts. A t-SNE projection of the penultimate layer of EfficientNetB3 for all 100 fine classes, colour-coded by class and annotated with class labels, would show whether the learned representation space respects fine-class structure or whether some classes remain entangled even after full fine-tuning.

The ViT family changes this bias profile. A transformer can model global relationships through self-attention, but it does not provide the same built-in locality and translation-equivariance prior as a CNN. This makes from-scratch ViT training on CIFAR-100 more demanding. In this project, the final transformer benchmark was therefore not a randomly initialised ViT, but a pretrained Hugging Face ViT-B/16 adapted to CIFAR-100 fine labels. This distinction is important: the strong ViT result reflects transformer transfer learning and parameter-efficient adaptation, not evidence that a small ViT trained from scratch is naturally superior on 32×32 images.

Chapter 7

Conclusions

This project benchmarked deep learning architectures for binary and multiclass image classification on CIFAR-100, comparing from-scratch baselines against transfer-learning models across recurrent, convolutional, and transformer-based designs.

From-scratch models provide a valuable control layer. The baseline CNN (macro F1 0.2629 on coarse multiclass) and strong CNN with regularisation and augmentation (0.2739) establish the floor of what shallow networks trained on 50 000 small images can achieve. These controls contextualise the absolute performance of larger models and confirm that the bulk of modern image classification gains comes not from superior optimisation but from architectural capacity, inductive bias, and pretrained representation quality.

Among from-scratch controls, recurrent models proved competitive. In the project's recorded coarse multiclass run with the row-wise $(32, 96)$ sequence encoding, Bi-LSTM reached macro F1 0.3450 and LSTM 0.3105, both above the baseline CNN and matching or exceeding the strong regularised CNN's 0.2739 (Table 5.4). This result reflects the particular structure of 32×32 imagery under sequential encoding rather than any general superiority of recurrence over convolution: sequence models lack the spatial locality bias that makes CNNs the default for natural images at scale. However, the finding does illustrate that on low-resolution, fixed-size data with a fixed sequence length, attention to row-wise structure can encode useful context.

Transfer-learning models dominate across every metric. MobileNetV3Small unfrozen achieves macro F1 0.8409 on coarse multiclass (Table 5.5), more than double the best from-scratch result. EfficientNetB3 fine-tuned reaches 0.8319 on the harder 100-class fine task (Table 5.6). The strongest fine-label result comes from the standalone Hugging Face ViT-B/16 experiment: the LoRA-adapted ViT reaches macro F1 0.9165 and Top-5 accuracy 0.9919 on the 100-class fine task. Every pretrained backbone, whether frozen, fine-tuned, or adapted through parameter-efficient tuning, substantially outperforms every from-scratch model, confirming the empirical principle that large-scale pretraining transfers effectively to smaller downstream tasks, even across resolution shifts, framework differences, and label distribution changes.

The project highlights four practical lessons for image classification. First, representation matters more than raw model size: MobileNetV3Small, a lightweight mobile-friendly backbone, outperforms larger from-scratch CNNs by a margin of 0.55 macro F1 points when fine-tuned, demonstrating that pretrained feature hierarchies compress more signal per parameter than randomly initialised weights. Second, metric choice is critical: accuracy alone

is misleading on imbalanced tasks (e.g., the *cow* fine binary task scores 0.9867 accuracy while its F1 is only 0.2130). Macro F1 and ROC AUC provide fairer comparisons when classes vary in frequency. Third, fine-tuning consistently improves over frozen feature extraction: ResNet50V2 coarse results improve from 0.7832 to 0.8236, MobileNetV3Small from 0.7935 to 0.8409. Task-specific adaptation of pretrained weights is worth the additional compute when the target task has sufficient examples to avoid overfitting. Fourth, transformer results require careful framing: the ViT-B/16 result is the best fine-label result in the project, but it uses a standalone Hugging Face/PyTorch workflow, 224×224 preprocessing, and LoRA-style adaptation, so it should be interpreted as a transformer transfer-learning benchmark rather than a directly identical run to the local TensorFlow/Keras CNN pipeline.

On small natural images, representation and pretraining dominate raw architectural choice. The gap between pretrained and from-scratch models dwarfs the gap between convolutional, recurrent, and transformer mechanisms trained or adapted on the same dataset. This hierarchy is stable across task granularities: from coarse superclass discrimination to fine 100-class classification, transfer learning provides the largest performance gains, followed by architectural family within the from-scratch cohort. The ViT-B/16 LoRA result shows that transformer transfer learning can push fine-grained classification further, but it also shows why fair comparisons must account for preprocessing, pretraining scale, and adaptation strategy. Future work should focus on augmentation strategies, fine-tuning schedules, transformer ablations, and per-class error analysis to squeeze incremental gains; the architectural frontier in image classification is no longer only at the drawing board but in the details of pretraining, domain adaptation, and careful evaluation metric selection.

Bibliography

[1] Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*. Technical report, University of Toronto, 2009.

[2] Hochreiter, S. and Schmidhuber, J. *Long Short-Term Memory*. Neural Computation, 9(8), pp. 1735–1780, 1997.

[3] He, K., Zhang, X., Ren, S. and Sun, J. *Deep Residual Learning for Image Recognition*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[4] Huang, G., Liu, Z., van der Maaten, L. and Weinberger, K. Q. *Densely Connected Convolutional Networks*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[5] Tan, M. and Le, Q. V. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. In Proceedings of the International Conference on Machine Learning (ICML), 2019.

[6] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V. and Adam, H. *Searching for MobileNetV3*. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[7] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. and Houlsby, N. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. In International Conference on Learning Representations (ICLR), 2021.

[8] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. In International Conference on Learning Representations (ICLR), 2022.